[Institución]

Manual de usuario: inicio de sesión y registro

MacroEntreno — pruebas funcionales y automatizadas

Alexis Vallejos

Materia: [Materia]

Docente: [Docente]

Curso: [Curso]

Fecha: 30 de octubre de 2026

Resumen

Este manual explica cómo crear una cuenta, iniciar sesión y cerrar sesión en MacroEntreno, una aplicación móvil hecha con React Native y Expo que se conecta a una API en Node.js con base de datos MySQL. Cada función se muestra con capturas reales tomadas mientras se usaba la aplicación, incluidos los mensajes de error que ve el usuario cuando un dato está mal. Además se documentan las pruebas automatizadas que verifican el registro, el inicio de sesión y la seguridad del acceso, los problemas encontrados durante la práctica y cómo se resolvieron.

Palabras clave: inicio de sesión, registro, validación, JWT, pruebas automatizadas, React Native.

Índice

Introducción **4**
Desarrollo **5**
Entorno de prueba 5
Inicio de sesión 5
Pantalla de inicio de sesión 5
Validación de los campos 6
Credenciales incorrectas 7
Ingreso según el rol 8
Inicio de sesión con redes sociales 9
Registro de una cuenta nueva 10
Pantalla de registro 10
Validación de los campos 11
Email ya registrado 13
Creación de la cuenta 13
Perfil, cierre de sesión y nuevo ingreso 14
Seguridad del acceso 17
Configuración del asistente con IA 19
Pruebas automatizadas 20
Procedimiento 20
Resultados 21
Problemas encontrados y soluciones 21
Limitaciones 22
Conclusión **24**
Referencias **25**

Introducción

MacroEntreno sirve para registrar comidas, macronutrientes y entrenamientos de gimnasio, con seguimiento de un entrenador. Tiene tres roles: **Atleta**, **Entrenador** y **Administrador**. Para usar cualquier pantalla primero hay que identificarse, por eso el inicio de sesión y el registro son la puerta de entrada de toda la aplicación.

El objetivo del trabajo fue comprobar que esas dos funciones andan bien y que no se puede entrar sin pasar por las validaciones. Para eso se hicieron dos tipos de pruebas: pruebas manuales recorriendo la aplicación como lo haría un usuario, con capturas de cada paso, y pruebas automatizadas que envían datos correctos e incorrectos al servidor y revisan cada respuesta. El manual describe primero cómo se usa cada pantalla y después qué se probó y qué resultados se obtuvieron.

Desarrollo

Entorno de prueba

Las pruebas se hicieron en una computadora con macOS. La API se levantó con `npm run dev:memory`, que crea una base MySQL 8 temporal y carga usuarios de ejemplo; la aplicación se abrió en su versión web con `npx expo start --web`, en una ventana del tamaño de un celular (390 × 844 píxeles). La versión web usa las mismas pantallas y el mismo código que la del teléfono; la diferencia es dónde se guarda la sesión, como se explica más adelante.

Las capturas se tomaron con Playwright, una herramienta que maneja el navegador de forma automática: escribe en los campos, toca los botones y guarda una imagen de la pantalla en cada paso (Microsoft, s.f.). Los usuarios de ejemplo que trae la base se resumen en la Tabla 1.

Tabla 1

Usuarios de ejemplo usados en las pruebas

Email	Contraseña	Rol	Pantalla de inicio
admin@macroentreno.com	Demo1234	Administrador	Panel
coach@macroentreno.com	Demo1234	Entrenador	Mis atletas
atleta@macroentreno.com	Demo1234	Atleta	Inicio
lucia.gomez@mail.com	Lucia2026	Atleta	Inicio (cuenta creada en la prueba)

Nota. Las tres primeras cuentas las crea el script de la base de datos. La cuarta se creó durante la prueba de registro.

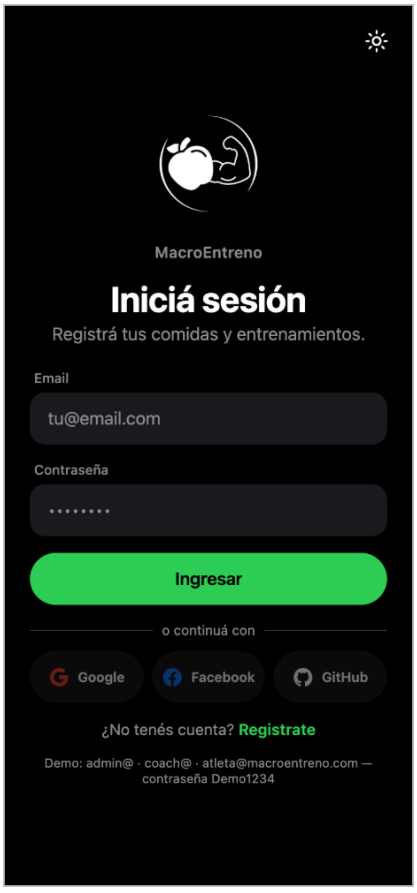
Inicio de sesión

Pantalla de inicio de sesión

Al abrir la aplicación sin una sesión guardada aparece la pantalla de la Figura 1. Tiene dos campos, **Email** y **Contraseña**, el botón **Ingresar**, los botones de redes sociales (Google, Facebook y GitHub) y el enlace **Registrate** para crear una cuenta nueva. Abajo se recuerdan las cuentas de ejemplo. El ícono del sol, arriba a la derecha, cambia entre tema claro y oscuro.

Figura 1

Pantalla de inicio de sesión de MacroEntreno



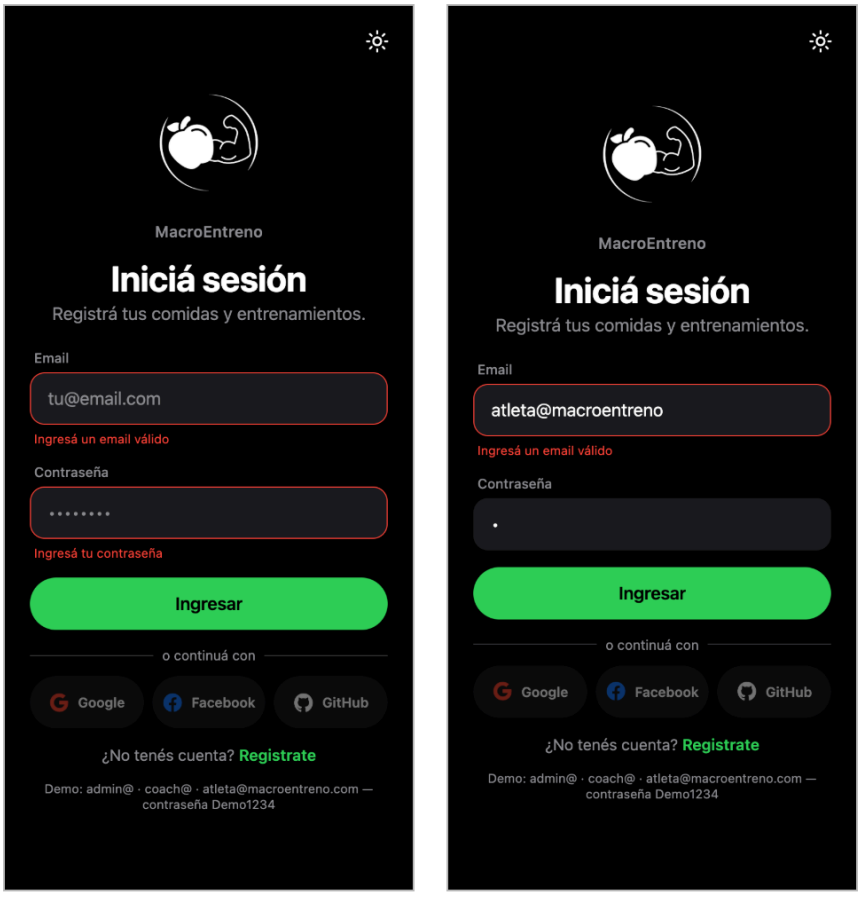
Para ingresar se escribe el email, se pasa a la contraseña (la tecla Siguiente del teclado lleva directo a ese campo), se escribe la contraseña y se toca **Ingresar** o la tecla Ir del teclado. Mientras el servidor responde, el botón muestra un indicador de carga y no se puede volver a tocar.

Validación de los campos

Antes de enviar nada al servidor, la aplicación revisa los datos. Si se toca **Ingresar** con los campos vacíos, cada campo se marca en rojo con su mensaje debajo, como muestra la Figura 2a. Si el email no tiene el formato `nombre@dominio.algo`, aparece "Ingresá un email válido" (Figura 2b). Al empezar a escribir en un campo, su error desaparece.

Figura 2

Validación del formulario de inicio de sesión



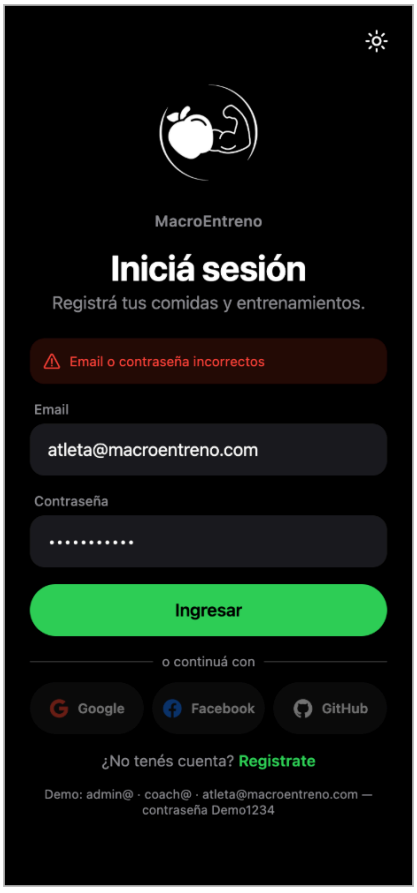
(a) Campos vacíos (b) Email sin dominio completo

Credenciales incorrectas

Si el email tiene buen formato pero la contraseña no es la correcta, el servidor responde con el código HTTP 401 y la aplicación muestra el recuadro rojo de la Figura 3: "Email o contraseña incorrectos". Se probó con la cuenta `atleta@macroentreno.com` y la contraseña `Incorrecta1`.

Figura 3

Mensaje de error ante una contraseña incorrecta



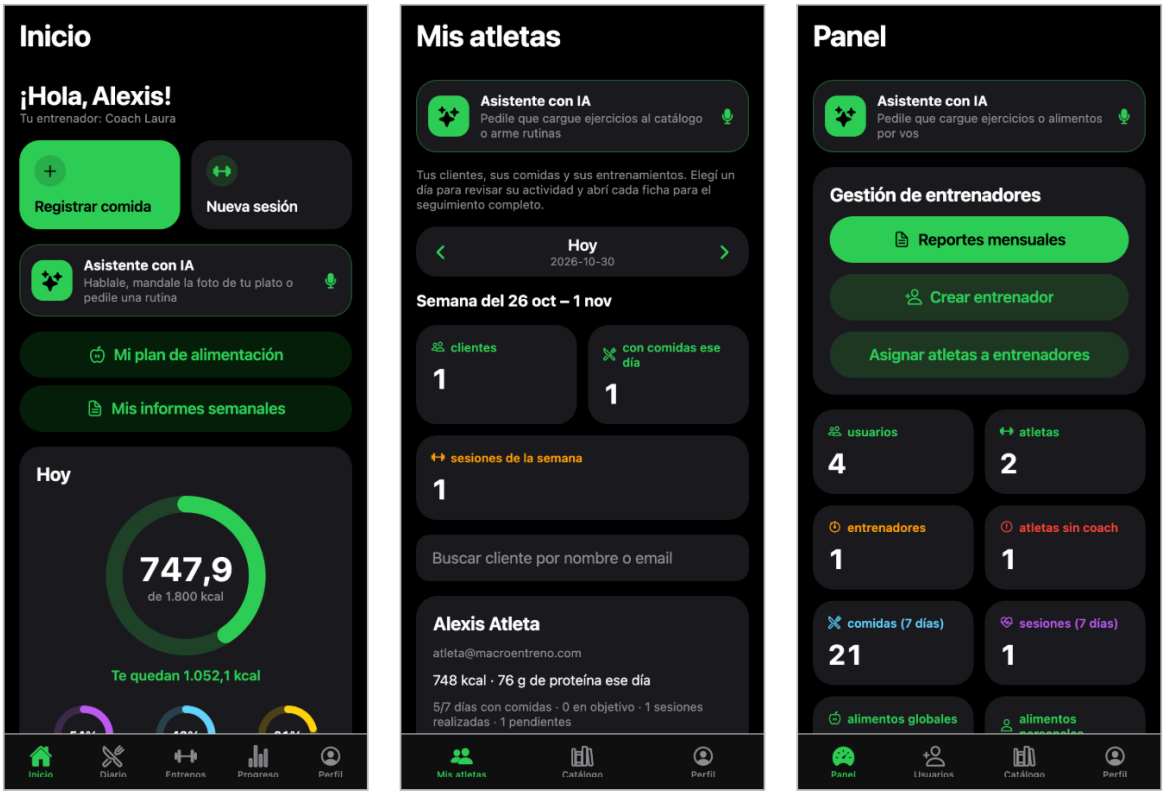
El mensaje es el mismo cuando el email no existe. Esto es a propósito: si dijera "ese email no está registrado", cualquiera podría averiguar qué personas tienen cuenta probando emails. La guía de autenticación de OWASP recomienda responder igual en los dos casos (OWASP Foundation, s.f.). Las pruebas automatizadas comprueban que las dos respuestas son idénticas.

Ingreso según el rol

Con datos correctos la aplicación entra y muestra pestañas distintas según el rol de la cuenta. En la Figura 4 se ven las tres pantallas obtenidas al ingresar con cada usuario de la Tabla 1: el Atleta ve su resumen diario con calorías y accesos para registrar comidas y sesiones; el Entrenador ve la lista de sus atletas; el Administrador ve el panel con estadísticas y la gestión de entrenadores. La navegación usa rutas protegidas de Expo Router: sin sesión solo se puede llegar al inicio de sesión y al registro, y cada pestaña se habilita según el rol (Expo, s.f.-a).

Figura 4

Pantalla inicial según el rol de la cuenta



(a) Atleta (b) Entrenador (c) Administrador

Al ingresar, el servidor entrega dos tokens JWT: uno de acceso, que dura 15 minutos y se envía en cada pedido, y uno de renovación, que dura 30 días y sirve para pedir un token de acceso nuevo sin volver a escribir la contraseña (Jones et al., 2015). Cada renovación invalida el token anterior; si alguien intenta reutilizar uno ya usado, el servidor cierra todas las sesiones de esa cuenta.

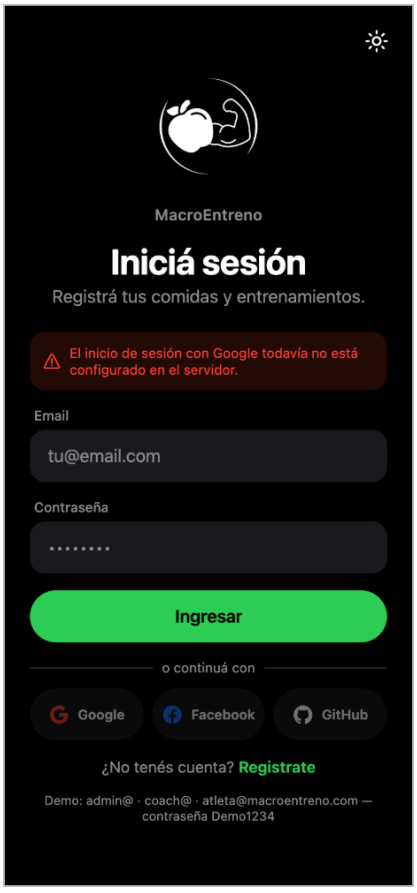
Inicio de sesión con redes sociales

Los botones Google, Facebook y GitHub abren el navegador para que el usuario autorice a MacroEntreno con su cuenta de esa red. El intercambio se hace en el servidor con OAuth 2.0 y PKCE, un mecanismo que evita que otra aplicación del teléfono pueda robar el código de autorización (Hardt, 2012; Sakimura et al., 2015).

En el entorno de prueba no había credenciales de ninguna red social cargadas en el servidor. Por eso, al tocar Google, la aplicación avisa que ese inicio de sesión todavía no está configurado (Figura 5) en lugar de abrir una página que daría error. El ingreso real con Google, Facebook o GitHub no se pudo probar de forma manual; lo que sí se verificó es el flujo completo con un proveedor simulado en las pruebas automatizadas del servidor.

Figura 5

Aviso al usar una red social que el servidor no tiene configurada



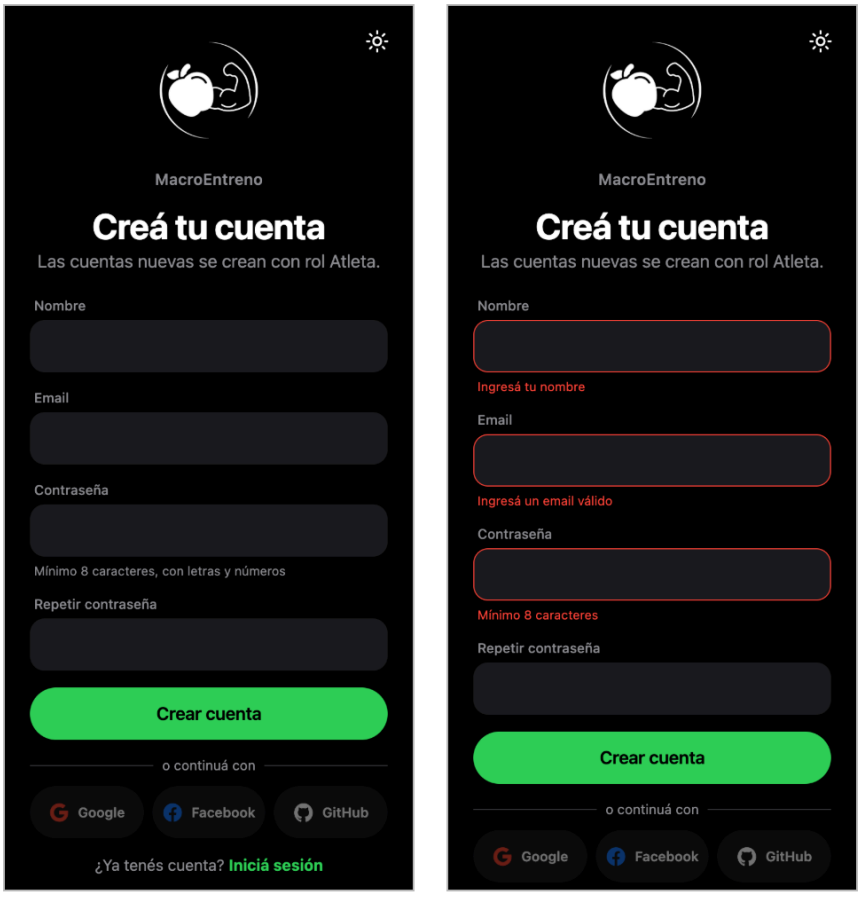
Registro de una cuenta nueva

Pantalla de registro

Desde el enlace **Registrate** se llega a la pantalla **Creá tu cuenta** (Figura 6a). Pide nombre, email, contraseña y repetir la contraseña. Debajo del campo de contraseña se indica la regla: mínimo 8 caracteres, con letras y números. Toda cuenta creada desde acá tiene rol Atleta; los roles de Entrenador y Administrador solo los puede asignar un administrador.

Figura 6

Pantalla de registro vacía y con los campos sin completar



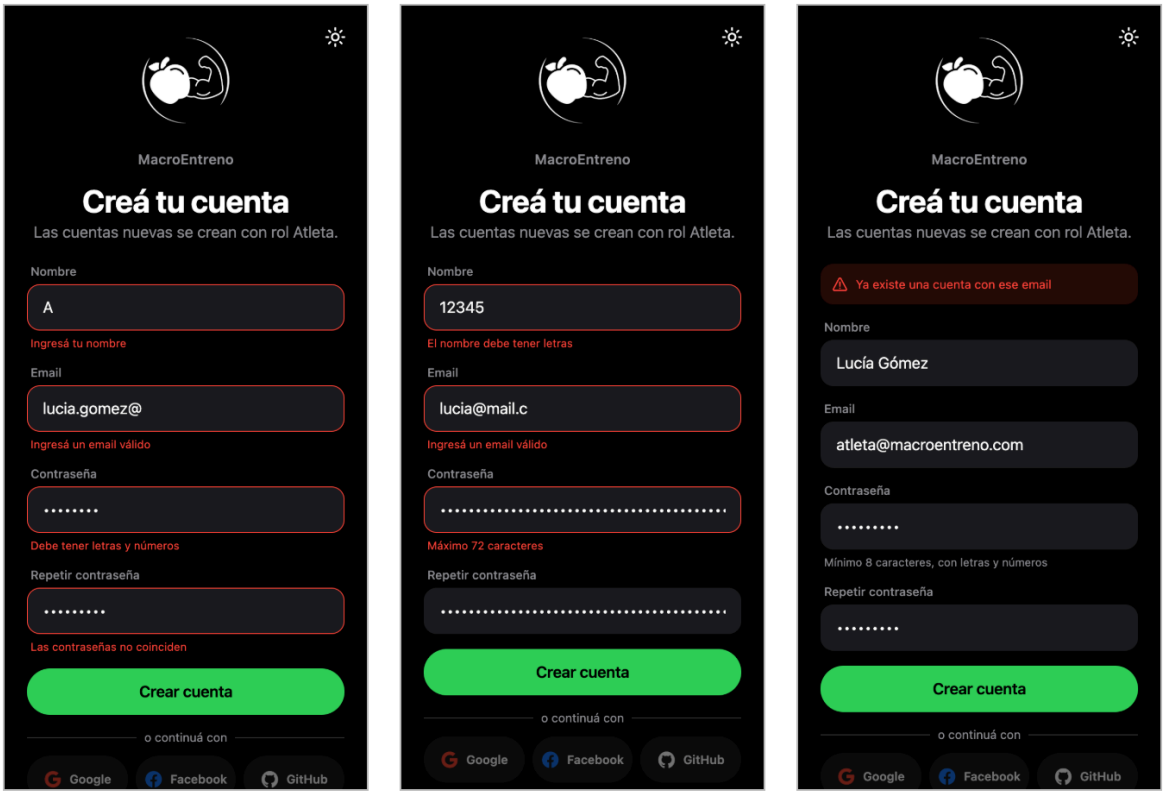
(a) Pantalla inicial (b) Al tocar Crear cuenta vacío

Validación de los campos

Si se toca **Crear cuenta** sin completar nada, se marcan nombre, email y contraseña (Figura 6b). En la Figura 7a se cargaron a propósito datos incorrectos en todos los campos: un nombre de una sola letra, un email sin dominio, una contraseña sin números y una confirmación distinta. Cada campo muestra su propio mensaje, así el usuario sabe exactamente qué corregir. Las reglas completas se resumen en la Tabla 2.

Figura 7

Errores de validación del registro y email ya registrado



(a) Datos incorrectos (b) Límites de cada campo (c) Email que ya tiene cuenta

En la Figura 7b se probaron los límites: un nombre formado solo por números ("12345"), un email cuya extensión tiene una sola letra (`lucia@mail.c`) y una contraseña de 74 caracteres. La aplicación los rechaza antes de enviarlos, con los mismos criterios que aplica el servidor.

Tabla 2

Reglas de validación del registro y del inicio de sesión

Campo	Regla	Mensaje en la aplicación
Nombre	Entre 2 y 100 caracteres sin contar espacios al principio o al final	"Ingresá tu nombre" / "Máximo 100 caracteres"
Nombre	Al menos una letra; sin caracteres de control ni invisibles	"El nombre debe tener letras" / "El nombre tiene caracteres no permitidos"
Email	Formato nombre@dominio.ext, extensión de 2 letras o más, solo caracteres ASCII	"Ingresá un email válido"
Email	Máximo 190 caracteres (largo de la columna en la base)	"Máximo 190 caracteres"
Contraseña	Entre 8 y 72 caracteres, con al menos una letra y un número	"Mínimo 8 caracteres" / "Máximo 72 caracteres" / "Debe tener letras y números"
Contraseña	No más de 72 bytes: las tildes, la ñ y los emojis ocupan más de uno	"Es muy larga: las tildes, la ñ y los emojis ocupan más espacio"
Repetir contraseña	Igual a la contraseña, letra por letra	"Las contraseñas no coinciden"
Contraseña (login)	No puede estar vacía ni superar 72 bytes	"Ingresá tu contraseña" / "Máximo 72 caracteres"

Nota. Las mismas reglas se aplican en la aplicación (`mobile/src/utils/validation.js`) y en el servidor (`backend/src/utils/validation.js`). La aplicación avisa antes de enviar; el servidor vuelve a controlar todo por si alguien envía pedidos sin usar la aplicación.

Email ya registrado

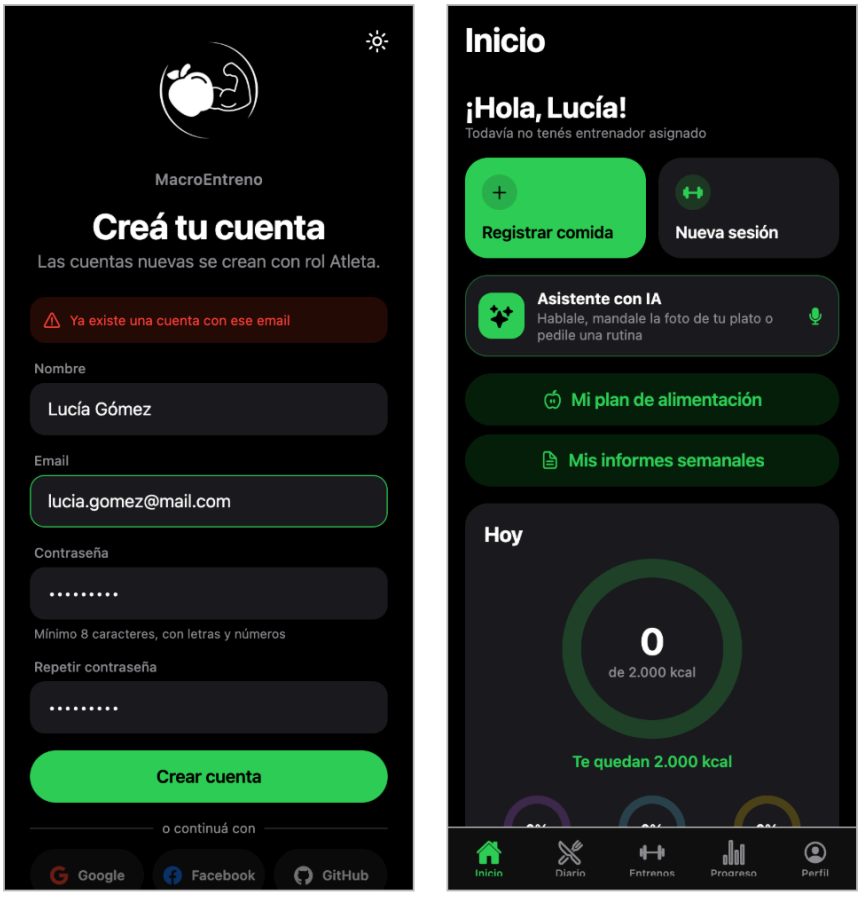
La Figura 7c muestra qué pasa al intentar registrarse con `atleta@macroentreno.-com`, que ya pertenece a otra cuenta: el servidor responde 409 (conflicto) y la aplicación muestra "Ya existe una cuenta con ese email". No importa si el email se escribe con mayúsculas o espacios: el servidor lo pasa a minúsculas y le quita los espacios antes de compararlo, así que no se pueden crear dos cuentas con el mismo email escrito distinto.

Creación de la cuenta

Con los datos corregidos (Figura 8a) se tocó **Crear cuenta**. La cuenta se creó y la aplicación entró directamente, sin pedir que se inicie sesión otra vez (Figura 8b). El saludo usa el nombre cargado y se indica que todavía no tiene entrenador asignado. La contraseña se guarda en la base cifrada con bcrypt, nunca en texto plano.

Figura 8

Registro completo y primera pantalla de la cuenta nueva



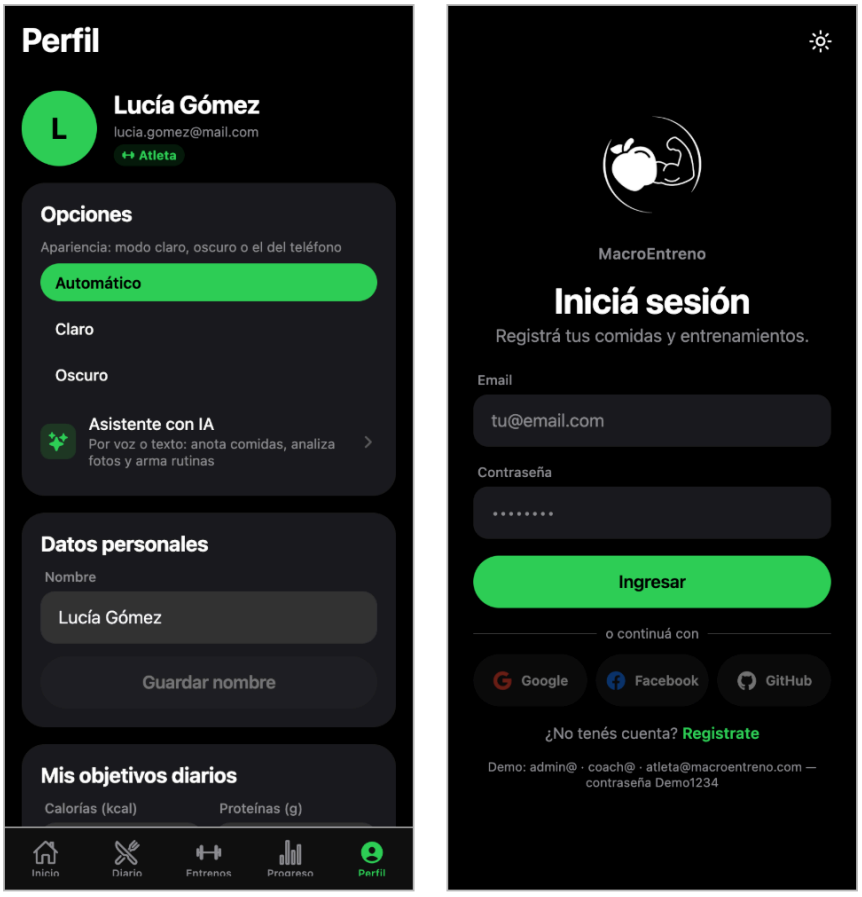
(a) Formulario válido (b) Cuenta creada

Perfil, cierre de sesión y nuevo ingreso

En la pestaña **Perfil** se ven el nombre, el email y el rol de la cuenta (Figura 9a). Al final de esa pantalla está **Cerrar sesión**; la aplicación pide confirmar y, al aceptar, borra los tokens guardados y vuelve al inicio de sesión (Figura 9b).

Figura 9

Perfil de la cuenta nueva y cierre de sesión

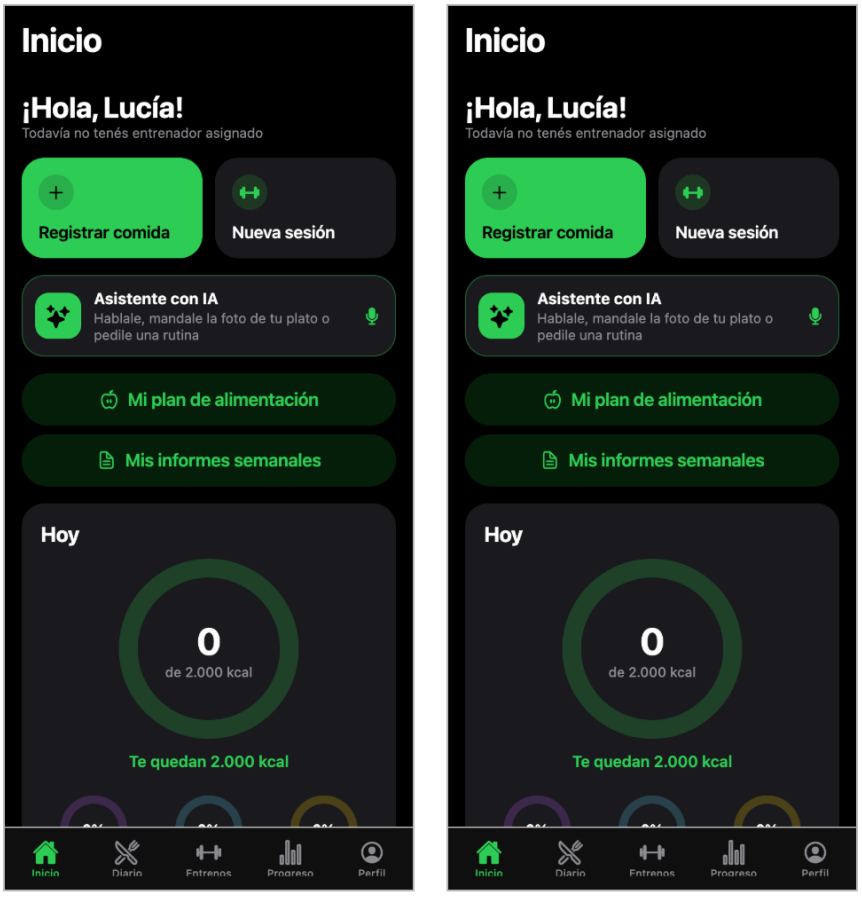


(a) Perfil (b) Después de cerrar sesión

Después se volvió a ingresar con la cuenta recién creada, `lucia.gomez@mail.com`, y la contraseña elegida en el registro (Figura 10a). Por último se recargó la página: la aplicación restauró la sesión sola, sin pedir los datos de nuevo (Figura 10b). En el teléfono los tokens se guardan en SecureStore, que usa el almacenamiento cifrado del sistema (Keychain en iOS, Keystore en Android); en la versión web se usa el almacenamiento del navegador, que solo está pensado para pruebas (Expo, s.f.-b).

Figura 10

Nuevo ingreso con la cuenta registrada y sesión restaurada al recargar

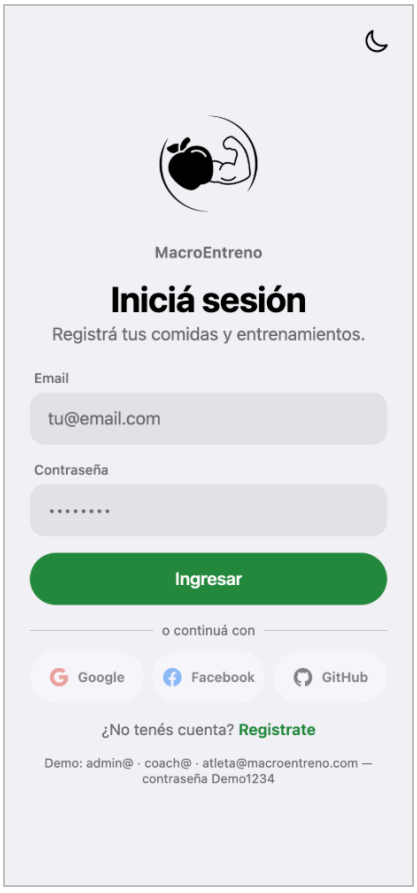


(a) Ingreso con la cuenta nueva (b) Después de recargar

La aplicación también respeta el tema del sistema. La Figura 11 muestra la misma pantalla de inicio de sesión cuando el teléfono está en modo claro.

Figura 11

Inicio de sesión con el tema claro



Seguridad del acceso

Las validaciones de la aplicación ayudan al usuario, pero no alcanzan para proteger las cuentas: cualquiera puede enviar pedidos directo a la API sin pasar por las pantallas. Por eso se revisó cada forma de entrar (registro, inicio de sesión, renovación del token, cierre de sesión, cambio de contraseña y redes sociales) intentando saltear los controles. La Tabla 3 resume qué se intentó y qué responde ahora el servidor.

Tabla 3

Intentos de entrar sin pasar la validación y respuesta del servidor

Intento	Respuesta	Estado antes de la revisión
Token JWT sin firma (`alg: none`) o firmado con otro secreto	401 TOKEN_INVALID	Ya se rechazaba
Token válido modificado para decir rol "admin"	401; y un token que dice admin sobre un atleta no da permisos	Ya se rechazaba: el rol se lee de la base
Token firmado con otro algoritmo (HS512)	401	Corregido: se fijó HS256
Usar el token de renovación como token de acceso	401 TOKEN_INVALID	Ya se rechazaba
Token de un usuario desactivado o borrado	403 / 401	Ya se rechazaba
Inyección SQL en email o contraseña (por ejemplo `' OR 1=1 --`)	400 o 401, nunca entra	Ya era seguro: consultas parametrizadas
Objetos en vez de texto (`{"$ne": null}`)	400 VALIDATION_ERROR	Ya se rechazaba
Contraseña de 72 caracteres + texto extra	400; antes iniciaba sesión	Corregido: tope de 72 bytes
Contraseña corta en caracteres pero de más de 72 bytes (ñ, emojis)	400	Corregido
Registrarse enviando `role: "admin"` o `isActive`	Se ignora: la cuenta es Atleta	Ya era seguro
Email de más de 190 caracteres	400; antes error 500	Corregido
Nombre con caracteres invisibles, NUL o sin letras	400	Corregido
Cuerpo de más de 100 kB	413; antes error 500	Corregido
Adivinar la contraseña actual en Cambiar contraseña	Límite de 50 intentos cada 15 minutos	Corregido: no tenía límite
Canje de OAuth sin el `code_verifier` correcto	401 y el código queda inutilizado	Ya era seguro (PKCE)

Nota. Cada fila tiene al menos una prueba automática en `backend/test/auth.test.js`, dentro del grupo "seguridad: intentos de saltear la validación".

El caso más importante fue el de la contraseña larga. bcrypt, el algoritmo que cifra las contraseñas, solo usa los primeros 72 bytes y descarta el resto sin avisar. Se comprobó que, con una cuenta de contraseña de 72 caracteres, el inicio de sesión aceptaba esa contraseña con cualquier texto agregado al final. Ahora el servidor rechaza todo lo que pase de 72 bytes, tanto al registrarse como al iniciar sesión y al cambiar la contraseña.

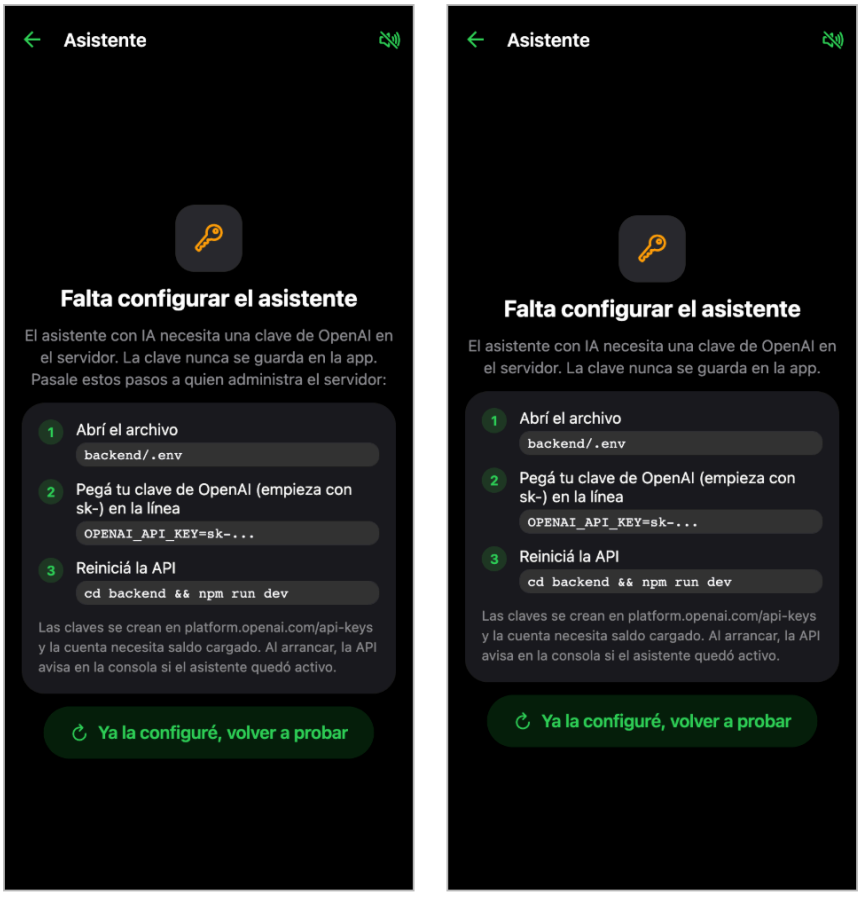
Además, el servidor limita a 50 los intentos de registro, inicio de sesión y cambio de contraseña cada 15 minutos por dirección IP. Si se supera, responde "Demasiados intentos, probá de nuevo en unos minutos". Esto frena a quien quiera probar contraseñas de forma automática.

Configuración del asistente con IA

El asistente con IA usa la API de OpenAI y necesita una clave que se carga **solo en el servidor**; la aplicación nunca la ve ni la guarda. Si la clave falta o es inválida, al abrir el asistente aparece el panel de la Figura 12 con los tres pasos para configurarlo, en lugar de un error genérico. Al usuario Atleta, además, se le indica que le pase esos pasos a quien administra el servidor.

Figura 12

Panel del asistente cuando el servidor no tiene la clave de OpenAI



(a) Usuario Atleta (b) Usuario Administrador

Para activarlo se siguen estos pasos:

1. Crear una clave en `https://platform.openai.com/api-keys` (empieza con `sk-`). La cuenta de OpenAI necesita saldo cargado.
2. Abrir el archivo `backend/.env` y pegar la clave después del signo igual, sin comillas, en la línea marcada entre flechas: `OPENAI_API_KEY=sk-...`
3. Reiniciar la API con `cd backend && npm run dev`. Al arrancar, la consola muestra "Asistente con IA: activo con el modelo …"; si falta la clave muestra "DESACTIVADO (falta OPENAI_API_KEY)".
4. En la aplicación, tocar **Ya la configuré, volver a probar**.

El archivo `backend/.env` está excluido de Git, así que la clave no se sube al repositorio. Si OpenAI rechaza la clave, se queda sin saldo, tarda demasiado o no responde, la aplicación muestra un mensaje claro en castellano para cada caso y un botón **Reintentar** en el mensaje que falló.

Pruebas automatizadas

Procedimiento

Además de las pruebas manuales se escribieron pruebas automáticas con el ejecutor de pruebas incluido en Node.js (`node:test`) y la librería Supertest, que envía pedidos HTTP a la API sin necesidad de abrir la aplicación (Node.js, s.f.). Cada archivo de pruebas crea su propia base MySQL temporal, carga los usuarios de ejemplo y la borra al terminar, de modo que las pruebas no dependen de datos anteriores.

Se agregaron dos archivos nuevos: `backend/test/auth.test.js`, con las pruebas del registro, el inicio de sesión y la seguridad del acceso, y `mobile/test/validation.test.mjs`, que prueba las reglas de validación de la aplicación. Para correrlas se usan estos comandos:

```
cd backend && npm test   # todas las pruebas del servidor
cd mobile  && npm test   # validaciones de la aplicación
```

Tabla 4

Grupos de pruebas automatizadas del inicio de sesión y el registro

Grupo de pruebas	Casos	Qué verifica
registro: alta exitosa	8	La cuenta se crea como Atleta, el email se guarda en minúsculas, la contraseña se guarda cifrada y el token sirve
registro: validaciones por campo	13	Cada regla de nombre, email y contraseña devuelve 400 con el campo que falló y no crea el usuario
registro: email duplicado	3	No se puede repetir un email, aunque cambien las mayúsculas o haya espacios
login: credenciales correctas	6	Ingreso de cada rol y normalización del email
login: credenciales incorrectas	11	Contraseña errónea, email inexistente, cuenta solo social y cuenta desactivada
flujo completo	13	Registro, ingreso, renovación del token, cierre de sesión y cambio de contraseña encadenados
seguridad: intentos de saltear la validación	39	Los intentos de la Tabla 3: tokens falsos, inyección, contraseñas largas, límites y OAuth
Validaciones de la aplicación (`validation.test.mjs`)	53	Las reglas de la Tabla 2 del lado del cliente, incluidos caracteres invisibles y límites

Nota. La cantidad de casos se tomó de la salida real de las pruebas. Los nombres completos de cada caso figuran en los archivos de prueba.

Resultados

Todas las pruebas pasaron. La Figura 13 muestra la ejecución del archivo de pruebas de autenticación: los siete grupos terminaron bien y el resumen indica 93 casos aprobados y ninguno fallido.

Figura 13

Salida de las pruebas de inicio de sesión y registro del servidor

```
$ cd backend && node --test test/auth.test.js
✔ registro: alta exitosa (13544.251711ms)
✔ registro: validaciones por campo (83.933757ms)
✔ registro: email duplicado (243.077946ms)
✔ login: credenciales correctas (1129.423087ms)
✔ login: credenciales incorrectas (832.466519ms)
✔ flujo completo: registro, login, refresh, logout y cambio de contraseña (2656.737055ms)
✔ seguridad: intentos de saltear la validación (4315.204219ms)
ℹ tests 93
ℹ suites 7
ℹ pass 93
ℹ fail 0
```

Para comprobar que los cambios no rompieron otras partes de la aplicación se corrieron también todas las pruebas del servidor y las de la aplicación (Figura 14). El servidor pasó 184 de 184 casos en 25 grupos, que incluyen roles, comidas, entrenamientos y el asistente con IA; la aplicación pasó 53 de 53. El analizador de código (`npx expo lint`) terminó sin errores ni advertencias.

Figura 14

Resumen de todas las pruebas del servidor y de la aplicación

```
$ cd backend && npm test
ℹ tests 184
ℹ suites 25
ℹ pass 184
ℹ fail 0
$ cd mobile && npm test
✔ isEmail (7.561825ms)
✔ validateLogin (5.332005ms)
✔ validatePassword (1.786337ms)
✔ validateRegister (5.776795ms)
✔ seguridad: intentos de saltear la validación del cliente (4.4163ms)
ℹ tests 53
ℹ suites 5
ℹ pass 53
ℹ fail 0
```

Problemas encontrados y soluciones

Durante la práctica aparecieron varios problemas. Se describen con el orden problema, causa, solución y resultado.

Email con espacios rechazado. Una prueba enviaba `" atleta@macroentreno.com "` y el servidor respondía 400 en lugar de dejar entrar. La causa era el orden de la validación con la librería Zod: primero revisaba el formato y después quitaba los espacios. Se cambió

para que primero recorte y pase a minúsculas, y después valide (Zod, s.f.). La prueba pasó a dar 200.

Contraseña más larga que lo que usa bcrypt. Al probar los límites se vio que una contraseña de 72 caracteres seguida de cualquier texto también iniciaba sesión, porque bcrypt ignora lo que pasa de 72 bytes. Se agregó un tope de 72 bytes en el registro, el inicio de sesión y el cambio de contraseña, en el servidor y en la aplicación. Ahora ese intento devuelve 400.

Errores 500 con datos raros. Un email de más de 190 caracteres, un cuerpo de más de 100 kB o un `refreshToken` numérico hacían fallar al servidor con error interno. Se agregaron las validaciones que faltaban y ahora responden 400 o 413 con un mensaje claro.

La aplicación no encontraba la API. En la primera prueba manual todos los pedidos fallaban con "No se pudo conectar con el servidor". Al revisar la consola del navegador se vio que la aplicación llamaba al puerto 3002, configurado en el archivo local `mobile/.env.local`, mientras la API escuchaba en el 3000. Para no modificar la configuración del equipo se redirigió el puerto 3002 al 3000 durante la prueba.

Cambios que no aparecían en la aplicación. Después de corregir las validaciones, una segunda tanda de capturas mostraba los mensajes del servidor y no los de la aplicación, señal de que el navegador seguía usando el código anterior. Se reinició el empaquetador de Expo limpiando su caché (`npx expo start --clear`) y se repitieron todas las capturas. Las que aparecen en este manual corresponden al código final.

Pantallas duplicadas para el navegador automático. Expo Router deja montada la pantalla anterior debajo de la nueva, así que al pasar al registro había dos campos "Email" en la página y Playwright no sabía cuál usar. Se resolvió indicándole que trabaje solo con los elementos visibles.

Limitaciones

- Las pruebas manuales se hicieron en la versión web de la aplicación, en un tamaño de pantalla de celular. No se instaló la APK en un teléfono Android ni se probó en un iPhone real, donde la sesión se guarda en SecureStore en lugar del navegador.
- El inicio de sesión con Google, Facebook y GitHub no se probó con cuentas reales porque el servidor de prueba no tenía credenciales de esas redes. El flujo se verificó con un proveedor simulado en las pruebas automatizadas.
- El asistente con IA no se probó con una clave real de OpenAI; se verificó el panel de configuración y, con respuestas simuladas, el manejo de cada error.
- Facebook no informa si el email de la cuenta está verificado. El servidor confía en que la API de Facebook solo devuelve emails confirmados al vincular cuentas por email; esto queda como un riesgo conocido.
- Al cambiar la contraseña se cierran las sesiones anteriores. Si otro dispositivo intenta renovar su token viejo, el servidor lo toma como reutilización y cierra

también la sesión nueva. Es un comportamiento seguro pero puede resultar molesto, y no se modificó.

Conclusión

El inicio de sesión y el registro de MacroEntreno funcionan como se esperaba: se crearon cuentas, se ingresó con los tres roles, se cerró la sesión y se volvió a entrar, y cada error mostró un mensaje que le dice al usuario qué corregir. Las pruebas automatizadas confirmaron lo mismo con 93 casos sobre la autenticación y con el resto de la aplicación sin fallas.

Lo más importante que dejó la práctica es que validar en la aplicación no alcanza: hay que validar de nuevo en el servidor, porque cualquiera puede enviarle pedidos sin pasar por las pantallas. Probar a propósito datos raros, como contraseñas muy largas, caracteres invisibles o tokens falsos, permitió encontrar fallas reales que con datos normales no aparecían, como el límite de 72 bytes de bcrypt.

También quedó claro que los problemas del entorno, como un puerto distinto o una caché vieja, pueden hacer creer que una función anda mal o que un arreglo ya está aplicado. Revisar la consola y repetir las pruebas después de cada cambio fue lo que permitió confirmar que las capturas del manual muestran el comportamiento real de la versión final.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

Expo. (s.f.-a). *Expo Router: Introduction*. Recuperado el 30 de octubre de 2026, de https://docs.expo.dev/router/introduction/

Expo. (s.f.-b). *SecureStore*. Recuperado el 30 de octubre de 2026, de https://docs.expo.dev/versions/latest/sdk/securestore/

Hardt, D. (Ed.). (2012). *The OAuth 2.0 authorization framework* (RFC 6749). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc6749

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519

Microsoft. (s.f.). *Playwright: Installation*. Recuperado el 30 de octubre de 2026, de https://playwright.dev/docs/intro

Node.js. (s.f.). *Test runner*. Recuperado el 30 de octubre de 2026, de https://nodejs.org/api/test.html

OWASP Foundation. (s.f.). *Authentication cheat sheet*. Recuperado el 30 de octubre de 2026, de https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Sakimura, N., Bradley, J., & Agarwal, N. (2015). *Proof key for code exchange by OAuth public clients* (RFC 7636). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7636

Zod. (s.f.). *Zod: TypeScript-first schema validation*. Recuperado el 30 de octubre de 2026, de https://zod.dev/